

Universidad Católica de Salta
Licenciatura en Ciencia de Datos

Trabajo Práctico N° 4

Programación II

Alumno: Facundo Luna

Profesor: Ing. Mario Martínez

Fecha: 12 de octubre de 2026

Índice

Aclaraciones previas **2**

1. Diccionario de stock **3**

2. Carga acumulativa de stock por consola **4**

3. Stock total de tornillos desde un archivo CSV **6**

4. Conteo de departamentos de 2 y 3 ambientes **7**

Matplotlib **9**

5. Gráfico de líneas de población histórica **10**

6. Gráfico multilínea de gastos mensuales **12**

7. Gráficos conjuntos «plot» y «scatter» **14**

8. Gráfico de barras de población histórica **16**

9. Gráfico de la parábola horizontal **18**

10. Gráfico de las funciones cuadrática y cúbica **20**

NumPy **22**

11. Suma, promedio y ordenamiento de un array **23**

12. Uso de where sobre un array **24**

13. Máscaras booleanas sobre un array **25**

Ejercicio final integrador **26**

14. Análisis anual de ventas y merma de un supermercado **27**

Aclaraciones previas

El presente trabajo práctico consiste en la codificación en Python de 14 ejercicios de la Unidad correspondiente de Programación II, referidos al uso de diccionarios, lectura de archivos CSV con el módulo `csv`, generación de gráficos con `matplotlib` y manipulación de arrays con `numpy`. Cada ejercicio se resuelve en un archivo independiente, nombrado según el formato exigido por la cátedra (`TP4_Ejercicio_XX.py`), incluyendo como primera línea un comentario con el apellido y nombre del alumno.

Los archivos de datos provistos por la cátedra se ubican en la misma carpeta que los programas, de modo que cada ejercicio abre `stock.csv`, `propiedades.csv` o `poblacion.csv` directamente por su nombre. El ejercicio final integrador incorpora además un archivo `ventas.csv` de elaboración propia.

Las figuras que acompañan a los ejercicios de `matplotlib` son la salida real de cada programa, obtenida al ejecutarlo.

El código fuente completo de cada ejercicio se encuentra disponible en el siguiente repositorio de GitHub, con el fin de facilitar su verificación y prueba:

https://github.com/Facugl/data-science-coursework/tree/main/primer-anio/programacion-2/tp-4/ejercicios

1. Diccionario de stock

Crear un diccionario vacío. El diccionario debe llamarse «stock». Luego de crear el diccionario complételo con el siguiente stock: tornillos = 100; tuercas = 150; arandelas = 300. Los nombres tornillos, tuercas y arandelas son las claves (keys) del diccionario mientras que las cantidades son los valores (values). Una vez armado el diccionario imprimirlo en pantalla con print.

Ejercicio en Python

```python
# Luna, Facundo

stock = {}

stock["tornillos"] = 100
stock["tuercas"] = 150
stock["arandelas"] = 300

print("Stock:", stock)
```

Código 1: Diccionario de stock

Resultado obtenido

```
{'tornillos': 100, 'tuercas': 150, 'arandelas': 300}
```

↑ Volver al índice

2. Carga acumulativa de stock por consola

Basado en el ejercicio anterior, utilizaremos el diccionario como una base de datos. Comenzaremos con un diccionario de stock de nuestros productos en cero: `stock = {'tornillos': 0, 'tuercas': 0, 'arandelas': 0}`*. Crear un bucle utilizando while que se ejecute de forma infinita* `while True`*. Dentro de ese bucle consultar al usuario por consola qué producto desea agregar al stock. Si el usuario ingresa «FIN» como producto se debe finalizar el bucle. Si el usuario ingresa un producto no definido en el stock se debe enviar un mensaje de error (esto se resuelve utilizando el operador «in» con diccionarios).*

Luego de haber ingresado el producto se debe ingresar por consola cuánto stock de ese producto se desea agregar al stock. Si teníamos 20 tornillos y el usuario desea agregar 10 tornillos más, en nuestro diccionario deben quedar 30 tornillos (debe acumular). Cuando el usuario ingrese «FIN» y se termine el bucle, debe imprimir en pantalla con print el diccionario con el stock final.

Ejercicio en Python

```python
# Luna, Facundo

stock = {'tornillos': 0, 'tuercas': 0, 'arandelas': 0}

while True:
    producto = input("Ingrese el producto a agregar al stock ('FIN' para terminar): ")

    if producto == "FIN":
        break

    if producto not in stock:
        print("Error: el producto no está definido en el stock")
        continue

    cantidad = int(input(f"Ingrese la cantidad de {producto} a agregar: "))
    stock[producto] = stock[producto] + cantidad

print("Stock:", stock)
```

Código 2: Carga acumulativa de stock por consola

Resultado obtenido

Ejemplo de ejecución cargando 20 y luego 10 tornillos, un producto inexistente y 5 tuercas:

```
Ingrese el producto a agregar al stock ('FIN' para terminar):
   tornillos
Ingrese la cantidad de tornillos a agregar: 20
Ingrese el producto a agregar al stock ('FIN' para terminar):
   tornillos
Ingrese la cantidad de tornillos a agregar: 10
Ingrese el producto a agregar al stock ('FIN' para terminar):
   clavos
Error: el producto no está definido en el stock
Ingrese el producto a agregar al stock ('FIN' para terminar):
   tuercas
Ingrese la cantidad de tuercas a agregar: 5
Ingrese el producto a agregar al stock ('FIN' para terminar): FIN
{'tornillos': 30, 'tuercas': 5, 'arandelas': 0}
```

↑ Volver al índice

3. Stock total de tornillos desde un archivo CSV

Realice un programa que abra el archivo «stock.csv» (se adjunta) en modo lectura y cuente el stock total de tornillos a lo largo de todo el archivo, sumando el stock en cada fila del archivo. Para eso debe leer los datos del archivo con «csv.DictReader», y luego recorrer los datos dentro de un bucle y solo acceder a la columna «tornillos» para cumplir con el enunciado del ejercicio.

Ejercicio en Python

```python
# Luna, Facundo

import csv

total_tornillos = 0

archivo = open("./stock.csv", "r")
datos = list(csv.DictReader(archivo))
archivo.close()

for fila in datos:
    total_tornillos = total_tornillos + int(fila["tornillos"])

print(f"El stock total de tornillos es: {total_tornillos}")
```

Código 3: Stock total de tornillos desde un archivo CSV

Resultado obtenido

```
El stock total de tornillos es: 445
```

↑ Volver al índice

4. Conteo de departamentos de 2 y 3 ambientes

Realice un programa que abra el archivo CSV «propiedades.csv» (se adjunta) en modo lectura. Recorrer dicho archivo y contar la cantidad de departamentos de 2 ambientes y la cantidad de departamentos de 3 ambientes disponibles. Al finalizar el proceso, imprima en pantalla los resultados. Tener cuidado que hay departamentos que no tienen definidos la cantidad de ambientes, verifique que el texto no esté vacío antes de convertirlo a entero con «int(..)». Si desea investigar puede evitar que el programa explote utilizando «try except».

Ejercicio en Python

De las 1049 filas del archivo, 7 tienen la columna `ambientes` vacía. Por eso el valor se guarda primero en una variable de texto y recién se convierte con `int()` después de comprobar que no esté vacía; cuando lo está, `continue` descarta la fila y el recorrido sigue con la siguiente. El `strip()` previo contempla el caso de que el campo traiga espacios en lugar de estar realmente vacío.

```python
# Luna, Facundo

import csv

departamentos_2_ambientes = 0
departamentos_3_ambientes = 0

archivo = open("propiedades.csv", "r", encoding="utf-8")
datos = list(csv.DictReader(archivo))
archivo.close()

for fila in datos:
    texto_ambientes = fila["ambientes"].strip()

    if texto_ambientes == "":
        continue

    ambientes = int(texto_ambientes)

    if ambientes == 2:
        departamentos_2_ambientes = departamentos_2_ambientes + 1
    elif ambientes == 3:
        departamentos_3_ambientes = departamentos_3_ambientes + 1

print(f"Departamentos de 2 ambientes:
   {departamentos_2_ambientes}")
```

```
26 print(f"Departamentos de 3 ambientes:
   {departamentos_3_ambientes}")
```

Código 4: Conteo de departamentos de 2 y 3 ambientes

Resultado obtenido

```
Departamentos de 2 ambientes: 382
Departamentos de 3 ambientes: 239
```

↑ Volver al índice

Matplotlib

A partir de este punto, los ejercicios requieren el uso de la biblioteca `matplotlib` para la construcción de gráficos. En todos los casos se trabaja con el módulo `pyplot`, importado con el alias habitual `plt`, y la figura se muestra en pantalla con `plt.show()`.

5. Gráfico de líneas de población histórica

Realizar un gráfico «plot» con: years como «x» y poblacion como «y»; con los siguientes datos: Years: 1900, 1970, 1990, 2000, 2020. Población: 1650, 3692, 5263, 6070, 7800.

Ejercicio en Python

```python
# Luna, Facundo

import matplotlib.pyplot as plt

years = [1900, 1970, 1990, 2000, 2020]
poblacion = [1650, 3692, 5263, 6070, 7800]

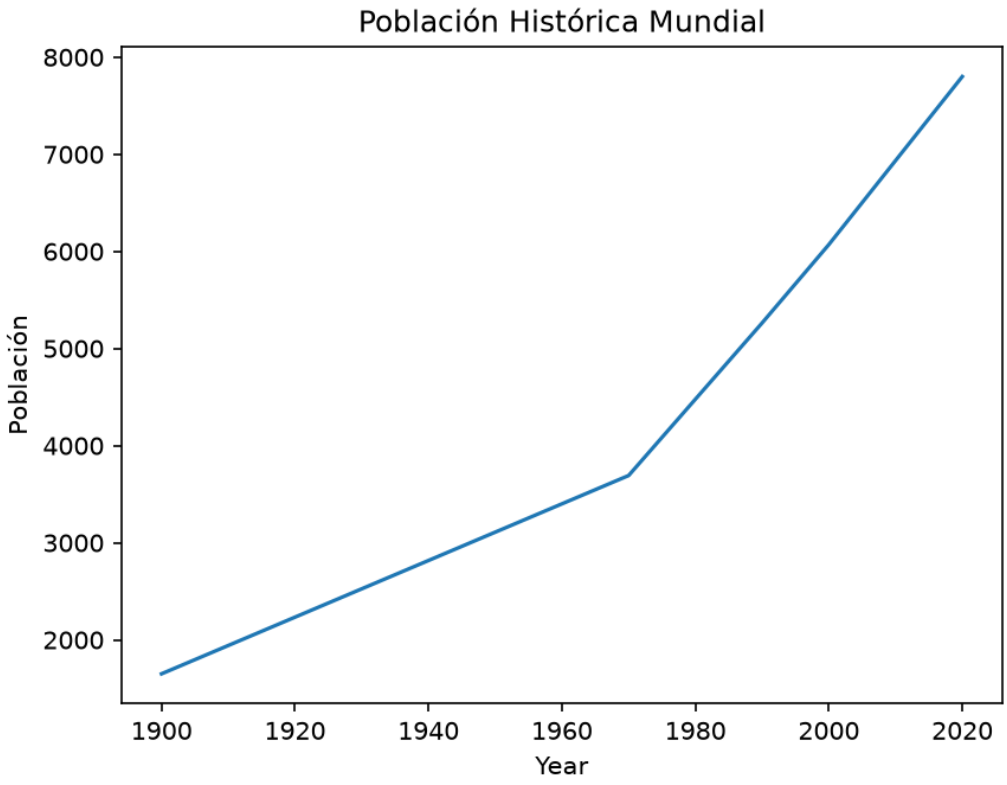
plt.plot(years, poblacion)

plt.title("Población Histórica Mundial")
plt.xlabel("Year")
plt.ylabel("Población")

plt.show()
```

Código 5: Gráfico de líneas de población histórica

Resultado obtenido



↑ Volver al índice

6. Gráfico multilínea de gastos mensuales

Realizar un gráfico «multi line plot». A partir de los siguientes datos: Mes: 3, 4, 5, 6; gasto_carne: 1650, 2600, 3100, 4000; gasto_verdura: 2500, 2200, 1800, 600. Calcular el gasto total como la suma de las listas gasto_carne y gasto_verdura. Realizar un gráfico «plot» con: mes como «x»; gasto_carne como «y1»; gasto_verdura como «y2»; gasto_total como «y3».

Ejercicio en Python

El gasto total se calcula recorriendo ambas listas con un bucle y sumando los valores correspondientes a un mismo mes, de modo que se obtiene una tercera lista de la misma longitud que las anteriores.

```python
# Luna, Facundo

import matplotlib.pyplot as plt

mes = [3, 4, 5, 6]
gasto_carne = [1650, 2600, 3100, 4000]
gasto_verdura = [2500, 2200, 1800, 600]

gasto_total = []

for i in range(len(mes)):
    gasto_total.append(gasto_carne[i] + gasto_verdura[i])

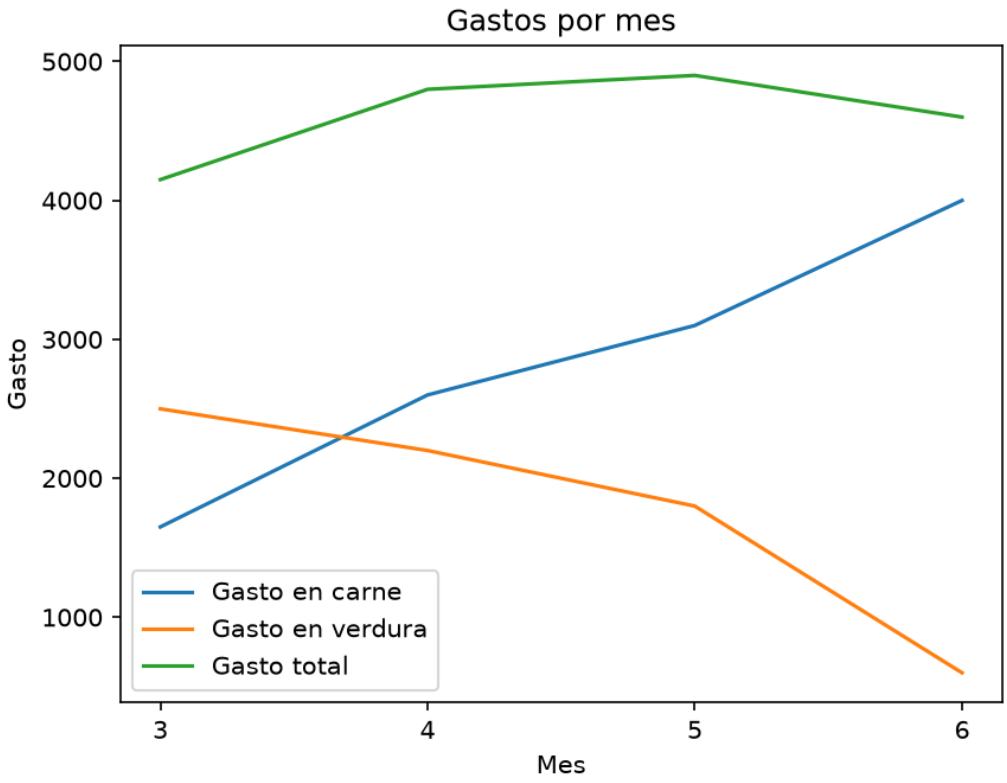
plt.plot(mes, gasto_carne, label="Gasto en carne")
plt.plot(mes, gasto_verdura, label="Gasto en verdura")
plt.plot(mes, gasto_total, label="Gasto total")

plt.title("Gastos por mes")
plt.xlabel("Mes")
plt.ylabel("Gasto")
plt.xticks(mes, ["Marzo", "Abril", "Mayo", "Junio"])
plt.legend()

plt.show()
```

Código 6: Gráfico multilínea de gastos mensuales

Resultado obtenido



↑ Volver al índice

7. Gráficos conjuntos «plot» y «scatter»

Realizar dos gráficos conjuntos «plot» y «scatter» (dentro de un mismo figure). Utilizar los datos del archivo poblacion.csv (adjunto). Year como «x» y población como «y». Título del gráfico «Población Histórica Mundial». Plot con líneas verdes; Scatter con puntos rojos.

Ejercicio en Python

Al llamar a `plt.plot()` y `plt.scatter()` de forma consecutiva, ambos gráficos se dibujan sobre la misma figura activa.

Las filas de `poblacion.csv` no vienen ordenadas por año, y `plot` une los puntos en el orden en que los recibe. Sin ordenar previamente, la línea retrocede dos veces sobre el eje horizontal y cruza todo el gráfico. Por eso los datos se ordenan por año antes de graficarlos. A `scatter` esto no lo afecta, porque dibuja cada punto en su posición sin unirlos entre sí.

```python
# Luna, Facundo

import csv
import matplotlib.pyplot as plt

years = []
poblacion = []

archivo = open("poblacion.csv", "r")
datos = list(csv.DictReader(archivo))
archivo.close()

datos.sort(key=lambda fila: int(fila["year"]))

for fila in datos:
    years.append(int(fila["year"]))
    poblacion.append(int(fila["poblacion"]))

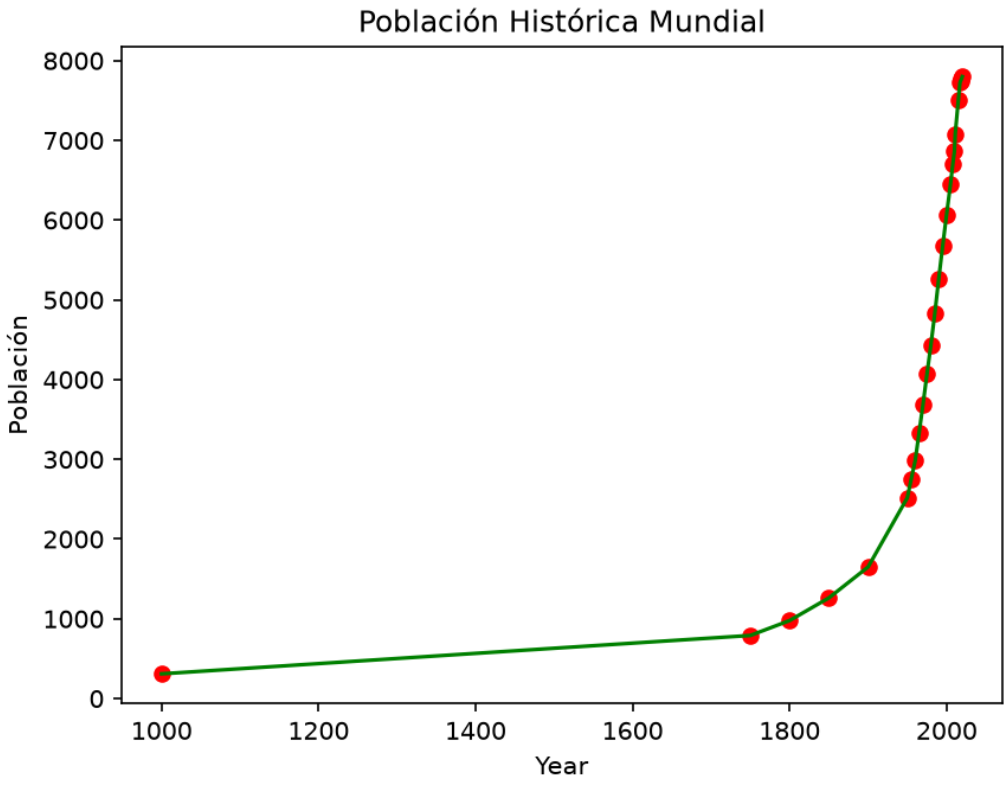
plt.plot(years, poblacion, color="green")
plt.scatter(years, poblacion, color="red")

plt.title("Población Histórica Mundial")
plt.xlabel("Year")
plt.ylabel("Población")

plt.show()
```

Código 7: Gráficos conjuntos plot y scatter

Resultado obtenido



↑ Volver al índice

8. Gráfico de barras de población histórica

Realizar un gráfico de barras. Utilizar los datos del archivo poblacion.csv (adjunto). Year como «x» y población como «y». Título del gráfico «Población Histórica Mundial». Visualizar solo los datos de los años 2000, 2005, 2010, 2015, 2020. Es decir, partiendo del csv mencionado, recorrerlo creando la estructura de datos necesaria para luego graficar la población de los años mencionados.

Ejercicio en Python

La estructura de datos elegida es un diccionario que vincula cada año buscado con su población. De esta forma el archivo se recorre una sola vez y luego los valores se extraen en el orden deseado, con independencia del orden en que aparecen en el CSV.

```python
# Luna, Facundo

import csv
import matplotlib.pyplot as plt

years_buscados = [2000, 2005, 2010, 2015, 2020]
poblacion_por_year = {}

archivo = open("poblacion.csv", "r")
datos = list(csv.DictReader(archivo))
archivo.close()

for fila in datos:
    year = int(fila["year"])

    if year in years_buscados:
        poblacion_por_year[year] = int(fila["poblacion"])

years = []
poblacion = []

for year in years_buscados:
    years.append(year)
    poblacion.append(poblacion_por_year[year])

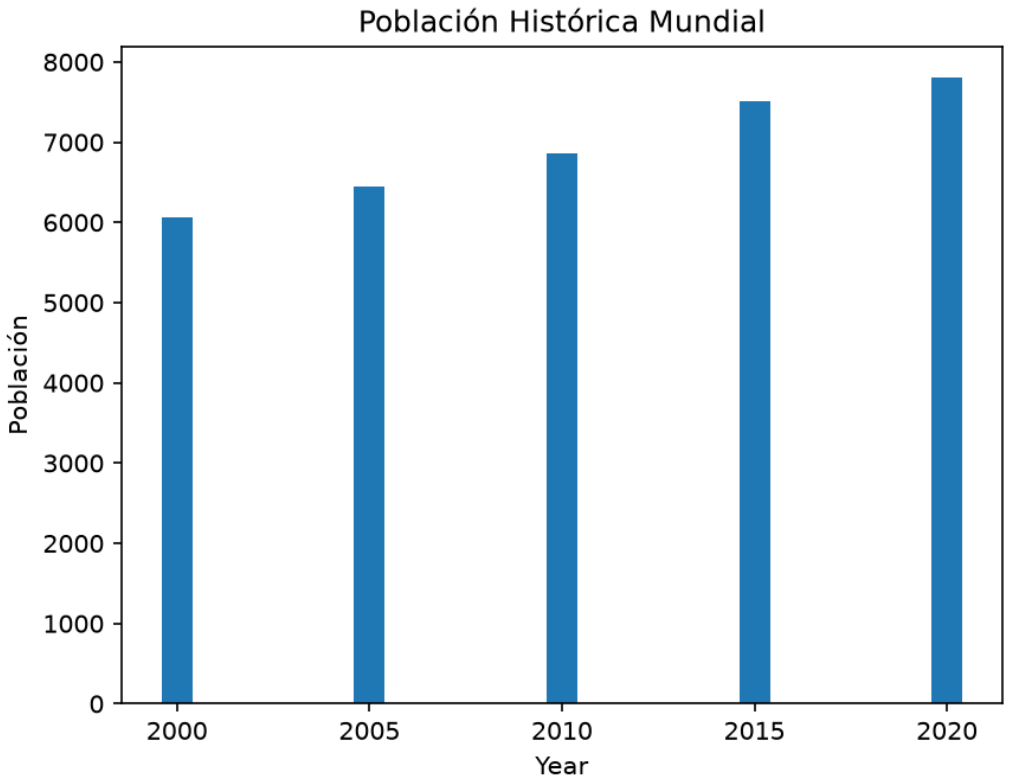
plt.bar(years, poblacion)

plt.title("Población Histórica Mundial")
plt.xlabel("Year")
```

```
30 plt.ylabel("Población")
31 plt.xticks(years)
32
33 plt.show()
```

Código 8: Gráfico de barras de población histórica

Resultado obtenido



↑ Volver al índice

9. Gráfico de la parábola horizontal

Realizar el código necesario para visualizar el gráfico propuesto en el enunciado: una parábola de eje horizontal, titulada «Valores x e y», con línea verde y la referencia «Ej_9».

Ejercicio en Python

La curva del enunciado corresponde a la relación $x = y^2$: el eje horizontal recorre valores de 0 a 100 mientras que el vertical va de -10 a 10. Por eso se genera primero el arreglo de valores de y con `np.linspace()` y a partir de él se calcula x, invirtiendo los roles habituales de ambos ejes al momento de graficar.

```python
# Luna, Facundo

import numpy as np
import matplotlib.pyplot as plt

y = np.linspace(-10, 10, 200)
x = y ** 2

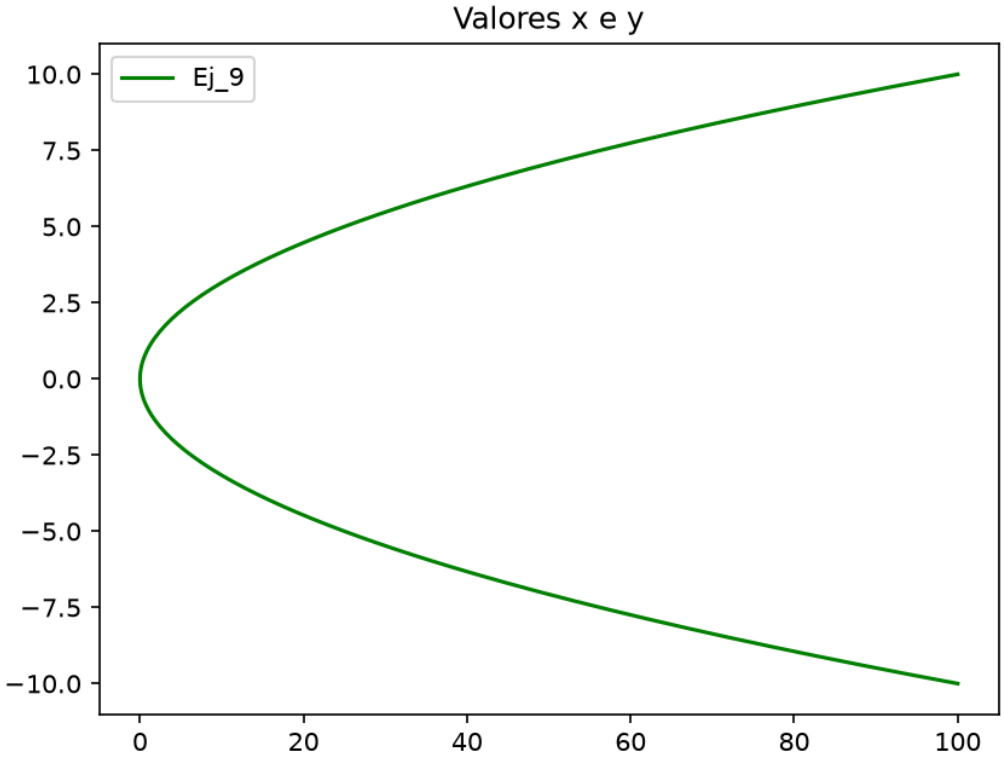
plt.plot(x, y, color="green", label="Ej_9")

plt.title("Valores x e y")
plt.legend()

plt.show()
```

Código 9: Gráfico de la parábola horizontal

Resultado obtenido



↑ Volver al índice

10. Gráfico de las funciones cuadrática y cúbica

*Realizar el código necesario para visualizar el gráfico propuesto en el enunciado: las curvas correspondientes a x^2 y x^3 en un mismo gráfico, titulado «Ejercicio_10», con las referencias «**2» y «**3».*

Ejercicio en Python

Ambas curvas se construyen sobre un mismo arreglo de valores de x entre -4 y 4, generado con `np.linspace()`, y se dibujan sobre la misma figura. La grilla se activa con `plt.grid(True)`.

```python
# Luna, Facundo

import numpy as np
import matplotlib.pyplot as plt

x = np.linspace(-4, 4, 200)
y1 = x ** 2
y2 = x ** 3

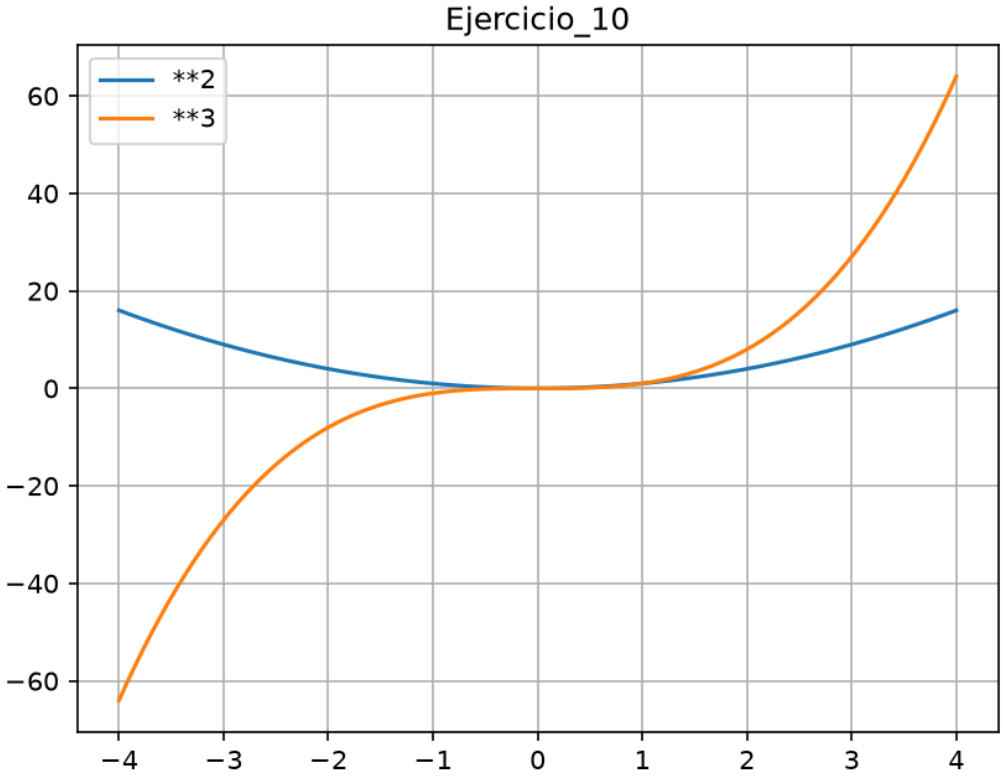
plt.plot(x, y1, label="**2")
plt.plot(x, y2, label="**3")

plt.title("Ejercicio_10")
plt.legend()
plt.grid(True)

plt.show()
```

Código 10: Gráfico de las funciones cuadrática y cúbica

Resultado obtenido



↑ Volver al índice

NumPy

Los ejercicios siguientes utilizan la biblioteca `numpy`, importada con el alias habitual `np`, para operar sobre arrays de forma vectorizada, es decir, aplicando una operación sobre todos los elementos del array sin necesidad de recorrerlo con un bucle.

11. Suma, promedio y ordenamiento de un array

Dado el siguiente array numpy: [1, 5, -2, 10, 2], realizar las siguientes operaciones: a. Suma; b. Promedio; c. Ordenarlo. Mostrar todos los resultados.

Ejercicio en Python

```python
# Luna, Facundo

import numpy as np

array = np.array([1, 5, -2, 10, 2])

suma = array.sum()
promedio = array.mean()
ordenado = np.sort(array)

print(f"Array original: {array}")
print(f"Suma: {suma}")
print(f"Promedio: {promedio}")
print(f"Ordenado: {ordenado}")
```

Código 11: Suma, promedio y ordenamiento de un array

Resultado obtenido

```
Array original: [ 1  5 -2 10  2]
Suma: 16
Promedio: 3.2
Ordenado: [-2  1  2  5 10]
```

↑ Volver al índice

12. Uso de where sobre un array

Crear un array a partir de una lista: `l1 = list(range(10))`*. Luego, usando where: a) Crear un nuevo array que solo tenga los números mayores a 3 del array numpy y los demás reemplazar por cero; b) Crear un nuevo array que solo tenga los números pares del array numpy y los demás reemplazar por cero. Mostrar todos los resultados.*

Ejercicio en Python

La función `np.where()` recibe una condición y dos valores alternativos: conserva el elemento del array cuando la condición es verdadera y lo reemplaza por cero cuando es falsa, devolviendo un array nuevo sin modificar el original.

```python
# Luna, Facundo

import numpy as np

l1 = list(range(10))
array = np.array(l1)

mayores_a_3 = np.where(array > 3, array, 0)
pares = np.where(array % 2 == 0, array, 0)

print(f"Array original: {array}")
print(f"Mayores a 3: {mayores_a_3}")
print(f"Pares: {pares}")
```

Código 12: Uso de where sobre un array

Resultado obtenido

```
Array original: [0 1 2 3 4 5 6 7 8 9]
Mayores a 3 (el resto en cero): [0 0 0 0 4 5 6 7 8 9]
Pares (el resto en cero): [0 0 2 0 4 0 6 0 8 0]
```

↑ Volver al índice

13. Máscaras booleanas sobre un array

Crear un array numpy: [1, 2, 4, 7]. Luego: a. Crear la máscara para solo quedarnos con los números mayores a 1; b. Crear la máscara para solo quedarnos con los números pares. Mostrar todos los resultados.

Ejercicio en Python

Una máscara es un array de valores booleanos, del mismo tamaño que el original, que resulta de aplicarle una condición. Al usar esa máscara como índice del array se obtienen únicamente los elementos que cumplen la condición. A diferencia de `np.where()`, aquí el array resultante es más corto que el original.

```python
# Luna, Facundo

import numpy as np

array = np.array([1, 2, 4, 7])

mascara_mayores_a_1 = array > 1
mascara_pares = array % 2 == 0

print(f"Array original: {array}")
print(f"Máscara de mayores a 1: {mascara_mayores_a_1}")
print(f"Números mayores a 1: {array[mascara_mayores_a_1]}")
print(f"Máscara de pares: {mascara_pares}")
print(f"Números pares: {array[mascara_pares]}")
```

Código 13: Máscaras booleanas sobre un array

Resultado obtenido

```
Array original: [1 2 4 7]
Máscara de mayores a 1: [False  True  True  True]
Números mayores a 1: [2 4 7]
Máscara de pares: [False  True  True False]
Números pares: [2 4]
```

↑ Volver al índice

Ejercicio final integrador

Inventar un ejercicio de enunciado propio y desarrollo individual, que contenga todo lo visto en la materia, al menos una vez cada herramienta vista. Es decir, la resolución del enunciado debe tener: cálculos secuenciales; ingreso de datos por parte del usuario, y mostrarle resultados; funciones para el tratamiento de cadenas de texto; estructuras de selección; bucles; diccionario de datos; lectura de un archivo csv; gráficos con Matplotlib; array NumPy.

14. Análisis anual de ventas y merma de un supermercado

Enunciado propuesto

Una sucursal de supermercado registra sus ventas del año en el archivo `ventas.csv`, con una fila por cada mes y sector del salón. El archivo tiene las columnas `mes`, `sector`, `unidades`, `precio_unitario` y `unidades_merma`, donde la merma son las unidades que se perdieron por vencimiento, rotura o deterioro y que, por lo tanto, nunca llegaron a venderse.

El control de gestión de la sucursal necesita un programa que:

a) Solicite el nombre del usuario y la sucursal, normalizando el texto ingresado.

b) Lea el archivo y calcule, para cada fila, la facturación como el producto entre las unidades vendidas y el precio unitario, y la merma en pesos como el producto entre las unidades perdidas y ese mismo precio.

c) Acumule en diccionarios la facturación y la merma anual de cada sector, y la facturación total de cada mes.

d) Muestre, sector por sector, la facturación, la merma en pesos y la merma como porcentaje de la facturación, señalando con la leyenda «ALERTA» aquellos sectores que superan el 3 % admitido por la cadena.

e) Solicite al usuario un sector e informe su porcentaje de merma, o un mensaje de error si el sector no existe.

f) Calcule, sobre un array NumPy con la facturación mensual, el total anual, el promedio mensual, el mejor y el peor mes, y determine qué meses superaron el promedio.

g) Grafique la evolución mensual de la facturación y la merma porcentual de cada sector.

Herramientas utilizadas

- **Cálculos secuenciales:** la facturación y la merma de cada fila, el porcentaje de merma por sector y el resumen anual.
- **Ingreso de datos y salida por pantalla:** el nombre del usuario, la sucursal y el sector a consultar, con sus respectivos `print`.
- **Tratamiento de cadenas:** `strip()`, `title()`, `upper()` y `capitalize()` para normalizar lo ingresado por el usuario y los valores leídos del archivo, y `ljust()` para alinear la tabla de resultados.

- **Estructuras de selección:** el control de claves ya existentes en los diccionarios, la comparación de la merma contra el límite admitido, el control de los datos vacíos ingresados por el usuario y la validación del sector consultado, mediante un `if` / `elif` / `else` con el operador `in`.
- **Bucles:** el recorrido del archivo y de los diccionarios con `for`, y la revalidación de los datos ingresados con `while`.
- **Diccionarios:** los acumuladores `ventas_por_sector`, `merma_por_sector` y `ventas_por_mes`, y el resultado del cálculo en `porcentajes_merma`.
- **Lectura de archivo CSV:** `csv.DictReader` sobre `ventas.csv`.
- **Gráficos con Matplotlib:** un gráfico de líneas con puntos y un gráfico de barras, dispuestos en una misma figura con `plt.subplots()`.
- **Array NumPy:** el array de facturación mensual, sobre el que se aplican `sum()`, `mean()`, `argmax()`, `argmin()`, `np.where()` y una división vectorizada para expresar los importes en millones.

Ejercicio en Python

```python
# Luna, Facundo

# EJERCICIO FINAL INTEGRADOR
# Analisis anual de ventas y merma de una sucursal de supermercado
#
# El archivo ventas.csv tiene una fila por cada mes y cada sector, con las unidades
# vendidas, el precio unitario y las unidades perdidas por vencimiento, rotura, etc
# El programa calcula la facturacion de cada sector,
# cuanto representa la merma sobre esa facturacion y como evoluciono la venta mes a mes

import csv
import numpy as np
import matplotlib.pyplot as plt

# Supongo hasta un 3% de merma por sector. Por encima de ese valor
# el encargado debe revisar el sector
MERMA_ACEPTABLE = 3.0

# El CSV identifica los meses por numero. Este diccionario traduce ese
# numero al nombre, para que los resultados se lean mejor
NOMBRES_MESES = {
    1: "Enero", 2: "Febrero", 3: "Marzo",
    4: "Abril", 5: "Mayo", 6: "Junio",
    7: "Julio", 8: "Agosto", 9: "Septiembre",
    10: "Octubre", 11: "Noviembre", 12: "Diciembre",
}


# Se eliminan espacios y se formatea el texto
# El while vuelve a preguntar mientras el dato quede vacio. Como strip() ya
# saco los espacios, escribir solo espacios tambien cuenta como vacio
nombre = ""

while nombre == "":
    nombre = input("Ingrese su nombre: ").strip().title()
```

```
36
37      if nombre == "":
38          print("El nombre no puede quedar vacío")
39
40  sucursal = ""
41
42  while sucursal == "":
43      sucursal = input("Ingrese la sucursal: ").strip().upper()
44
45      if sucursal == "":
46          print("La sucursal no puede quedar vacía")
47
48  print(f"Bienvenido/a al control de gestión, {nombre}")
49  print(f"Sucursal: {sucursal}")
50
51  ventas_por_sector = {}
52  merma_por_sector = {}
53  ventas_por_mes = {}
54
55
56  archivo = open("ventas.csv", "r")
57  datos = list(csv.DictReader(archivo))
58  archivo.close()
59
60
61  # El if/else resuelve el arranque de cada acumulador: si el
        sector todavia no
62  # figura en el diccionario no se le puede sumar nada, asi que la
        primera vez se
63  # asigna el valor y recien a partir de la segunda se acumula
64  for fila in datos:
65      mes = int(fila["mes"])
66      sector = fila["sector"].strip().capitalize()
67      unidades = int(fila["unidades"])
68      precio_unitario = float(fila["precio_unitario"])
69      unidades_merma = int(fila["unidades_merma"])
70
71      facturacion = unidades * precio_unitario
72      merma = unidades_merma * precio_unitario
73
74      if sector in ventas_por_sector:
75          ventas_por_sector[sector] = ventas_por_sector[sector] +
              facturacion
76          merma_por_sector[sector] = merma_por_sector[sector] +
              merma
```

```
77      else:
78          ventas_por_sector[sector] = facturacion
79          merma_por_sector[sector] = merma
80
81      if mes in ventas_por_mes:
82          ventas_por_mes[mes] = ventas_por_mes[mes] + facturacion
83      else:
84          ventas_por_mes[mes] = facturacion
85
86  # ljust() alinea los nombres a la izquierda y los :>15 alinean
        los numeros a la
87  # derecha, para que las columnas queden parejas en pantalla
88  print("\nFacturación y merma anual por sector:")
89  print(f"  {'Sector'.ljust(12)} {'Facturación':>15} {'Merma':>13}
        {'% merma':>8}   Estado")
90
91  porcentajes_merma = {}
92
93  for sector in ventas_por_sector:
94      facturacion = ventas_por_sector[sector]
95      merma = merma_por_sector[sector]
96      porcentaje = merma / facturacion * 100
97      porcentajes_merma[sector] = porcentaje
98
99      if porcentaje > MERMA_ACEPTABLE:
100         estado = "ALERTA"
101     else:
102         estado = "OK"
103
104     print(f"  {sector.ljust(12)} {facturacion:>15.2f}
            {merma:>13.2f} {porcentaje:>7.2f}%   {estado}")
105
106
107 # El operador in verifica que la clave exista antes de usarla,
        que es lo que
108 # evita que el programa corte con un error
109 #
110 # Esta consulta es opcional: sirve para mirar un sector en
        particular, pero el
111 # resumen anual y los graficos no dependen de ella. Por eso
        dejarla vacia no es
112 # un error, solo se informa que no se consulto nada y el
        programa sigue
113 consulta = input("\nIngrese un sector para consultar su merma: ")
114 consulta = consulta.strip().capitalize()
```

```
115
116 if consulta == "":
117     print("No se consultó ningún sector")
118 elif consulta in porcentajes_merma:
119     print(f"El sector {consulta} tuvo una merma del
        {porcentajes_merma[consulta]:.2f}%")
120 else:
121     print(f"Error: el sector {consulta} no figura en el archivo
        de ventas")
122
123
124 # sorted() devuelve los meses de menor a mayor, porque un
    diccionario no
125 # garantiza ningun orden. Con los importes en ese orden se arma
    el array
126 #
127 # sum() y mean() operan sobre todo el array sin necesidad de
    recorrerlo con un
128 # bucle. argmax() y argmin() no devuelven el importe sino la
    POSICION donde
129 # esta, y esa posicion se usa para saber de que mes se trata
130 meses = sorted(ventas_por_mes)
131
132 importes = []
133
134 for mes in meses:
135     importes.append(ventas_por_mes[mes])
136
137 importes = np.array(importes)
138
139 total_anual = importes.sum()
140 promedio_mensual = importes.mean()
141 mejor_mes = meses[int(importes.argmax())]
142 peor_mes = meses[int(importes.argmin())]
143
144 print("\nResumen anual de la sucursal:")
145 print(f"  Facturación total: ${total_anual:.2f}")
146 print(f"  Promedio mensual: ${promedio_mensual:.2f}")
147 print(f"  Mejor mes: {NOMBRES_MESES[mejor_mes]}")
148 print(f"  Peor mes: {NOMBRES_MESES[peor_mes]}")
149
150 # where() compara el array contra el promedio y arma uno nuevo
    del mismo largo:
151 # conserva el importe en los meses que lo superan y pone cero en
    el resto
```

```
# La division por un millon tambien se aplica a todo el array de una sola vez
sobre_promedio = np.where(importes > promedio_mensual, importes, 0)
sobre_promedio_en_millones = (sobre_promedio / 1000000).round(1)

# Los ceros que dejo where() son los meses que no superan el promedio,
# asi que el if los saltea y solo se listan los que si lo hacen
print("  Meses por encima del promedio:")

for i in range(len(meses)):
    if sobre_promedio_en_millones[i] > 0:
        nombre_mes = NOMBRES_MESES[meses[i]]
        millones = sobre_promedio_en_millones[i]
        print(f"    {nombre_mes.ljust(12)} ${millones:>5.1f} millones")

# subplots(1, 2) crea una figura con dos graficos lado a lado y devuelve uno
# por variable, de modo que cada uno se configura por separado
#
# El primero combina plot y scatter para ver la tendencia anual y marcar cada
# mes. set_xticks(meses) fuerza las marcas en los enteros 1 a 12, ya que por
# defecto matplotlib pondria valores intermedios como 2.5, que no existen
#
# El segundo es de barras y compara la merma de los sectores entre si. La
# rotacion de las etiquetas evita que los nombres se superpongan
#
# axhline() traza una linea horizontal a la altura que se le indique. Se usa
# la misma constante que decide el estado OK/ALERTA, asi que la linea siempre
# coincide con el limite real: las barras que la superan son las senaladas
figura, (grafico_meses, grafico_merma) = plt.subplots(1, 2, figsize=(12, 5))

grafico_meses.plot(meses, importes, color="green")
grafico_meses.scatter(meses, importes, color="red")
grafico_meses.set_title("Facturación mensual")
```

```
184 grafico_meses.set_xlabel("Mes")
185 grafico_meses.set_ylabel("Facturación")
186 grafico_meses.set_xticks(meses)
187 grafico_meses.grid(True)
188
189 etiqueta_limite = f"Límite admitido: {MERMA_ACEPTABLE}%"
190
191 grafico_merma.bar(list(porcentajes_merma),
       list(porcentajes_merma.values()))
192 grafico_merma.axhline(MERMA_ACEPTABLE, color="red",
       linestyle="--",
193                       label=etiqueta_limite)
194 grafico_merma.set_title("Merma anual por sector")
195 grafico_merma.set_xlabel("Sector")
196 grafico_merma.set_ylabel("Merma (%)")
197 grafico_merma.tick_params(axis="x", rotation=20)
198 grafico_merma.legend()
199
200 plt.show()
```

Código 14: Análisis anual de ventas y merma de un supermercado

Resultado obtenido

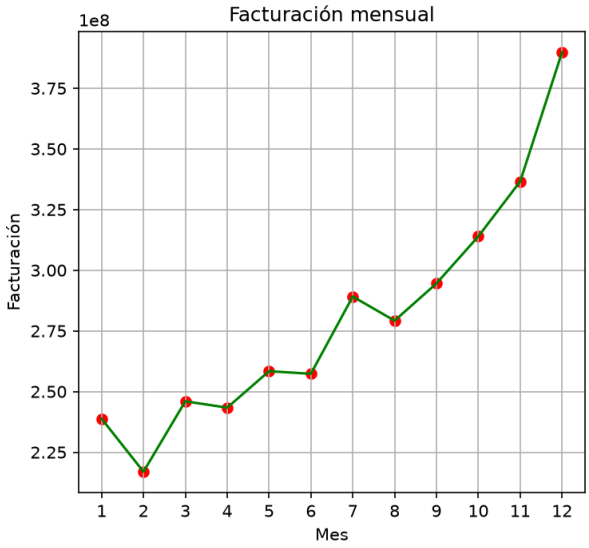
```
Ingrese su nombre: facundo luna
Ingrese la sucursal: sucursal norte
Bienvenido/a al control de gestión, Facundo Luna
Sucursal: SUCURSAL NORTE

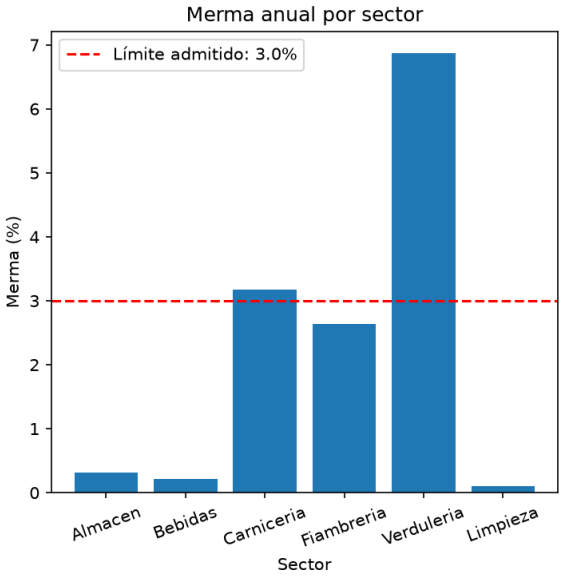
Facturación y merma anual por sector:
  Sector           Facturación          Merma   % merma    Estado
  Almacen         821457000.00     2603330.00     0.32%   OK
  Bebidas         492892200.00     1036510.00     0.21%   OK
  Carniceria      890182800.00    28222270.00     3.17%   ALERTA
  Fiambreria      438246600.00    11581900.00     2.64%   OK
  Verduleria      319560000.00    21949310.00     6.87%   ALERTA
  Limpieza        401770800.00      426420.00     0.11%   OK

Ingrese un sector para consultar su merma: carniceria
El sector Carniceria tuvo una merma del 3.17%

Resumen anual de la sucursal:
  Facturación total: $3364109400.00
  Promedio mensual: $280342450.00
  Mejor mes: Diciembre
  Peor mes: Febrero
  Meses por encima del promedio:
    Julio        $289.2 millones
    Septiembre   $294.6 millones
    Octubre      $313.9 millones
    Noviembre    $336.6 millones
    Diciembre    $389.8 millones
```

Los dos sectores señalados con «ALERTA» son los de productos frescos, que es donde la merma se concentra en la operación real de un supermercado.





↑ Volver al índice